BSc (Hons) in **Information Technology Specialized in AI**

Faculty of Computing

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Practical 03

> **Objective & Lecture Mapping:** In previous labs, we built a Simple Reflex Agent that moved randomly or reacted only to immediate adjacent cells. Today, we transition to a **Goal-Based/Planning Agent**. You will expose the environment's state space to the agent, allowing it to use **Breadth-First Search (BFS)**, **Depth-First Search (DFS)**, and **Uniform-Cost Search (UCS)** to simulate and find paths to the food before taking a physical action.

Part 1: Practical Implementation — Building the Search Agent

Step 1.1: Exposing the World Model

> Classical search algorithms require an abstract model of the environment to simulate future states. Currently, your agent is "blind" to the overall map.

1. Create a new Branch called Week_03 in your Forked Github Repo and work on the below Questions.
2. Open `visual_grid_game.py` .
3. Locate the `get_percept(self)` method.
4. Add three new keys to the dictionary to pass the global state to the agent:

```
'grid_size': (self.width, self.height)
'walls': list(self.walls)
'all_food': list(self.food_positions)
```

Step 1.2: Implementing BFS, DFS, and UCS

> You will implement three distinct uninformed search strategies. The core node expansion structure is identical; only the data structure for the **Frontier** changes!

1. Open `agent.py` and create a new class called `SearchAgent` .
2. Import required structures: `from collections import deque` and `import heapq` .
3. **BFS:** Create `def bfs_search(...)` . Use a FIFO queue (`deque.popleft()`). This explores the shallowest nodes first.
4. **DFS:** Create `def dfs_search(...)` . Use a LIFO stack (`list.pop()`). This explores the deepest nodes first.
5. **UCS:** Create `def ucs_search(...)` . Use a Priority Queue (`heapq.heappop()`) ordered by the total path cost $g(n)$.
6. For all three algorithms, you must maintain a `reached` set (or visited array) to track explored states. This converts your algorithm from a *Tree Search* to a *Graph Search*, preventing infinite loops.

Step 1.3: Executing and Comparing Offline Plans

> The agent must now form a complete plan and execute it step-by-step.

1. In your `SearchAgent.__init__` , add an empty list: `self.plan = []` and a config string `self.active_algo = 'BFS'` .
2. In `sense_and_act(self, percept)` , check if `self.plan` is empty.
3. If empty, find the closest food pellet from `percept['all_food']` , execute the search method matching `self.active_algo` , and store the resulting sequence of actions in `self.plan` .
4. Return the first action from the plan using `return self.plan.pop(0)` .
5. **Observation Task:** Change `self.active_algo` between 'BFS', 'DFS', and 'UCS' and run the simulation. Watch how DFS takes erratic, winding paths, while BFS and UCS take direct, optimal paths!

Part 2: Theoretical Evaluation

Based on your implementation and Lecture 04, complete the following analytical questions.

1. (Remember) You built your search logic based on the environment coordinates. What is the fundamental difference between the "State Space" and the "Search Tree"?

The state space is the actual set of possible world configurations. In this project, each reachable coordinate such as (x, y) is a state, subject to grid boundaries and walls. The state space is naturally a graph, because multiple different action sequences can reach the same coordinate.

The search tree is the structure created by expanding possible action sequences from the start. The same state may appear multiple times in the tree through different paths. For example, the coordinate (1, 1) might be reached by the action sequence Up --> Right, or by the action sequence Right --> Up. It is one state in the state space, but potentially two separate nodes in the search tree.

2. (Understand) In Step 1.2, you were instructed to use a `reached` set. In graph search theory, what specific problem does this set solve, and what would happen to your DFS agent without it?

The reached set converts tree search into graph search. It prevents the algorithm from:
* Revisiting the same coordinate repeatedly

* Expanding duplicate states

* Getting trapped in cycles

This is used in agent.py (lines 137-149) for BFS, and similarly for DFS. Without a reached set, the DFS agent could loop forever in a grid cycle; it could repeatedly generate the same coordinates instead of progressing toward the food. It would also waste memory by storing many duplicate paths to states it has already explored.

3. (Analyze) Compare the visual paths generated by BFS and DFS in your simulation. Why is BFS guaranteed to be optimal in this specific grid, whereas DFS produces winding, suboptimal routes?

BFS uses a FIFO queue and expands states level by level, as shown in agent.py (lines 137-149). It therefore finds all paths of length 1 before paths of length 2, then length 3, and so on. BFS is guaranteed to be optimal here because:

* Every movement action has the same cost: 1

* The grid is deterministic

* BFS tests paths in increasing depth order

Therefore, the first path that reaches a food pellet has the fewest moves, which makes it optimal.

DFS uses a LIFO stack, so it follows one branch as deeply as possible before trying alternatives. Its result depends heavily on the order in which neighbours are generated. It may explore a long route or a dead end before discovering a shorter route, which produces the winding, suboptimal behaviour observed in the simulation.

4. (Evaluate) If we scaled `visual_grid_game.py` up to a massive 1000x1000 grid, BFS and UCS might crash before finding food. According to the time and space complexity formulas from the lecture, contrast the memory bottlenecks of BFS versus DFS.

Let:
* b = branching factor
* d = depth of the shallowest solution
* m = maximum search depth
For tree search, the standard time and space complexities are:

Algorithm	Time	Space
BFS	O(b^d)	O(b^d)
DFS	O(b^m)	O(b·m)

BFS is the major memory risk because it stores an entire frontier of shallow nodes. This frontier can grow exponentially, O(b^d), before the food is reached.

DFS stores mainly the current path plus pending alternatives, so its frontier is much smaller, approximately O(b·m). However, this implementation uses graph search, so its reached set can still grow to the size of the explored state space, O(|V|).
A 1000×1000 grid can contain up to 1,000,000 coordinate states. Given this:

* BFS: frontier memory plus reached-set memory; it usually crashes from frontier growth first, since the frontier itself can approach the size of the whole grid.
* DFS: much lower frontier memory, but its reached set can still become very large as it explores the grid.
* UCS: similar memory concerns to BFS, with the added overhead of maintaining a priority queue.

The practical implementation also stores a complete action list with each frontier entry, which increases real memory usage beyond what the basic theoretical formulas suggest.

Once Completed Get a PDF of the completed File and Push into the Week 03 brach in the Github Project

Finalize and Lock Lab 03 Documentation